

Inside a neural network

From one adjustable calculation to a predictor of your cuboid's motion.

Prepared for Sarel Ye · 15 September 2026

The concrete idea: a neural network is a chain of numerical calculations. Some numbers enter as inputs, some are adjustable parameters, and the calculations produce predictions. Training adjusts the parameters; prediction uses their current values.

Start with the question your model must answer

"The cuboid has this velocity, and I apply this force. How far will it move, and how fast will it be moving after a specified interval?"

For your first transmodel, mass, friction, gravity, and the prediction interval stay fixed. The learned function receives velocity and force. It predicts displacement and final velocity, following your latest learning notes.

(velocity, force) → adjustable calculations → (displacement, final velocity)

How this supplement connects to Lesson 5

The original lesson introduces a $3 \rightarrow 4 \rightarrow 2$ network using position, velocity, and force. We will keep that exact structure to unpack its equations and its 26 parameters. Then we will connect it to your newer two-input prediction task.

This is a mathematical walkthrough, not implementation code. All example weights are invented for transparent arithmetic. **No network in this document has been trained.**

Your route through the explanation

One neuron → weights and bias → activation → a hidden layer → two outputs → the complete worked calculation → training and notebook checks.

A neuron is a small calculation

Multiply each input by a weight, add the results and a bias, then apply an activation function.

The word “neuron” is inspired by biology, but this mathematical unit is not a miniature brain. It does not read your physics description. In this network, it performs the following arithmetic:

$$q = w_x z_x + w_v z_v + w_F z_F + b$$

$$h = \varphi(q)$$

Symbol	Concrete meaning
z_x, z_v, z_F	Scaled position, velocity, and force supplied for one example.
w_x, w_v, w_F	Three adjustable weights: how each input contributes to this neuron's sum.
b	An adjustable bias: an offset added even when every input is zero.
q	The weighted sum before activation, called the pre-activation.
φ	The chosen activation function. The Greek letter is pronounced “phi.”
h	The neuron's output after activation; a calculated value, not a parameter.

Make the inputs concrete

For this illustration, divide position by 1 m, velocity by 1 m/s, and force by 10 N. A cuboid at $x = 0$ m, moving at $v = 0$ m/s, under $F = 10$ N becomes:

$$z = (z_x, z_v, z_F) = (0, 0, 1)$$

These are dimensionless inputs. Scaling changes their numerical representation, not the physical experiment. The original Lesson 5 also uses scaled inputs for its toy neuron.

Keep three things separate: inputs change between examples; weights and biases are adjusted by training; hidden values are recalculated whenever you make a prediction.

Vocabulary: neuron = 神经元; weight = 权重; bias = 偏置; activation function = 激活函数.

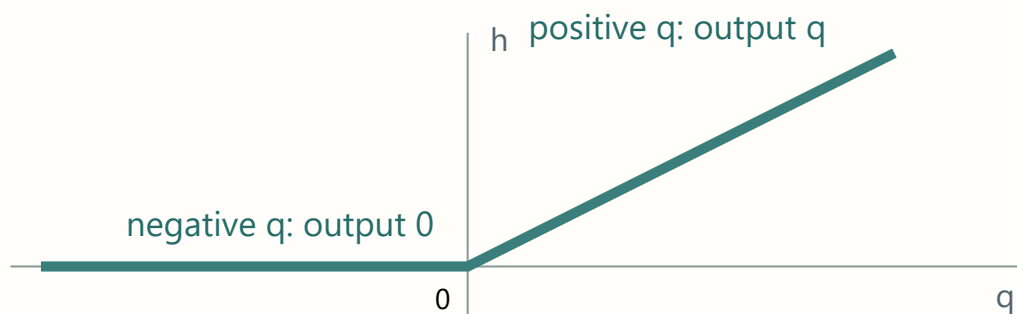
Follow the numbers

Use weights (0.2, -0.5, 0.8), bias -0.1, and input $z = (0, 0, 1)$. Before activation:

$$q = 0.2 \times 0 + (-0.5) \times 0 + 0.8 \times 1 - 0.1 = 0.7$$

Choose **ReLU**, the rectified linear unit. Its rule is simple: keep positive values and replace negative values with zero.

$$\text{ReLU}(q) = \max(0, q) \rightarrow h = 0.7$$



What does this neuron do as force changes?

Hold scaled position and velocity at zero. Then $q = 0.8z_F - 0.1$.

Physical force	Scaled force	q	$h = \text{ReLU}(q)$
0 N	0	-0.1	0
1.25 N	0.125	0	0
5 N	0.5	0.3	0.3
10 N	1	0.7	0.7

The force weight controls the slope of the active response. The bias shifts where activation begins: here, $z_F = 0.125$. Other input values also shift that boundary.

This is not a learned friction threshold. The 1.25 N boundary comes from invented weights. The guide's physical static-friction limit is 7.84 N. A hidden neuron's activation is not itself a statement that the cuboid moves.

With tanh instead of ReLU, the same $q = 0.7$ gives $h \approx 0.604$. tanh is a smooth alternative with outputs between -1 and 1; changing the activation changes the function family.

Several responses, one prediction

A hidden layer contains several neurons receiving the same inputs, each with its own weights and bias. They can therefore respond differently to the same situation.

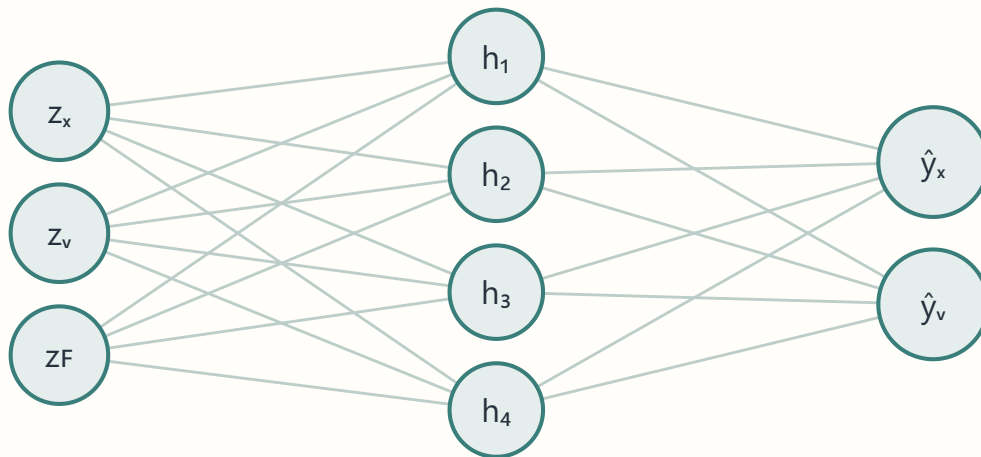


Figure 1. Each line carries a separate weight. Each hidden and output neuron also has a bias, omitted from the drawing. Input nodes simply hold supplied values.

Why “hidden”?

You supply the inputs and the correct final outputs, but you do not normally supply correct answers for h_1 through h_4 . These intermediate values are calculated inside the network. They are often called **features** or an internal representation.

One response might become useful for positive motion, another for a combination of force and velocity. But we cannot assume a neuron means “friction” or “acceleration.” Meaning depends on the learned network and may be distributed across many neurons.

How do four values become two outputs?

$$\begin{aligned}\hat{y}_x &= a_1h_1 + a_2h_2 + a_3h_3 + a_4h_4 + c_x \\ \hat{y}_v &= r_1h_1 + r_2h_2 + r_3h_3 + r_4h_4 + c_v\end{aligned}$$

Each output uses its own four weights and bias. Here we keep outputs linear, with no final ReLU, so predictions can be positive or negative even though the hidden ReLU values are nonnegative.

Calculate the entire network

Keep $z = (0, 0, 1)$. Use the original Lesson 5 neuron as neuron 1 and invent three more. “Forward pass” means calculating from inputs through hidden values to outputs, without changing parameters.

Neuron	Weights (x, v, F)	Bias	q for this input	ReLU(q)
1	(0.2, -0.5, 0.8)	-0.1	$0.8 - 0.1 = 0.7$	0.7
2	(0.1, 0.4, -0.3)	0.2	$-0.3 + 0.2 = -0.1$	0
3	(-0.2, 0.6, 0.5)	0.1	$0.5 + 0.1 = 0.6$	0.6
4	(0.3, -0.1, 0.2)	-0.4	$0.2 - 0.4 = -0.2$	0

Our hidden vector is therefore $\mathbf{h} = (0.7, 0, 0.6, 0)$. Now choose these illustrative output calculations:

$$\begin{aligned}\hat{y}_x &= 0.01h_1 + 0.02h_2 + 0.01h_3 - 0.01h_4 + 0 \\ \hat{y}_v &= 0.2h_1 - 0.1h_2 + 0.2h_3 + 0.1h_4 + 0\end{aligned}$$

Substituting the hidden values gives:

$$\begin{aligned}\hat{y}_x &= 0.007 + 0 + 0.006 - 0 = 0.013 \\ \hat{y}_v &= 0.14 - 0 + 0.12 + 0 = 0.26\end{aligned}$$

Interpret these as scaled next position and next velocity. With output scales 1 m and 1 m/s, they become **0.013 m and 0.26 m/s**. These output scales are chosen for this explanation, not recommended training settings.

Compare against the physical answer

Under Lesson 3's conditions (2 kg, $\mu_s = 0.4$, $\mu_k = 0.2$, $g = 9.8 \text{ m/s}^2$, 0.1 s), a 10 N push from rest gives $x_{\text{next}} = 0.0152 \text{ m}$ and $v_{\text{next}} = 0.304 \text{ m/s}$, starting at $x = 0$.

Quantity	Toy prediction	Reference label	Prediction minus label
Next position	0.013 m	0.0152 m	-0.0022 m
Next velocity	0.26 m/s	0.304 m/s	-0.044 m/s

You have now executed a neural network on paper. Its errors provide a training signal. These invented weights demonstrate arithmetic, not learned physical knowledge or accuracy across other cases.

Why not only weighted sums?

Without nonlinear activations, adding layers does not give this network a richer kind of function.

Consider two simple layers with no activation between them. If $h = 2u + 1$ and $y = 3h - 4$, substitution gives $y = 6u - 1$. Two layers have collapsed into one weighted sum plus a bias.

$$3(2u + 1) - 4 = 6u - 1$$

The same happens with vectors and matrices. A composition of affine transformations is still affine. "Affine" means a linear combination plus an offset; it need not pass through zero.

What ReLU adds

ReLU introduces a bend. Different neurons can bend at different input combinations. Weighted combinations of their responses can form a function with different slopes in different regions.

$$r(u) = \text{ReLU}(u - 1) - \text{ReLU}(u - 2)$$

Input region	Result	Reason
$u \leq 1$	$r(u) = 0$	Both terms are zero.
$1 < u < 2$	$r(u) = u - 1$	Only the first term is active.
$u \geq 2$	$r(u) = 1$	The two increasing terms cancel their slopes.

This small function is flat, then rises, then becomes flat again. One global affine function cannot reproduce those three regions.

Why this matters for the cuboid

Staying at rest, sliding positively, sliding negatively, and stopping within an interval require different physical relationships. Nonlinear networks can approximate richer behavior than one global affine mapping.

Flexibility is not a physics guarantee. A ReLU network does not automatically enforce static friction, exact rest, or correct stopping. Ordinary finite ReLU networks are continuous; ideal stick-slip rules may introduce sharp or even discontinuous changes. Errors near those boundaries need explicit evaluation.

At $q = 0$, ReLU has a corner. Training libraries use a defined derivative convention there. You do not need to solve that implementation detail to understand the forward calculation.

Three equations, now unpacked

$$q = W_1 z + b_1 \quad h = \varphi(q) \quad \hat{y} = W_2 h + b_2$$

A vector is an ordered list of numbers. A matrix is a rectangular table of numbers. Here, matrix multiplication is a compact way to calculate all the neurons' weighted sums at once.

Object	Shape	What it contains
z	3×1	The three scaled inputs, treated as a column.
W_1	4×3	One row of three weights per hidden neuron.
b_1	4×1	One bias per hidden neuron.
q, h	4×1 each	Four sums, then four activated values.
W_2	2×4	One row of four weights per output.
$b_2, \hat{y}$	2×1 each	Two output biases; two predictions.

For example, row 1 of W_1 is (0.2, -0.5, 0.8). Its multiplication with z is exactly the three multiplications and additions from page 3. Applying φ to q means applying the activation separately to each entry.

Where do the 26 parameters come from?

$$(4 \times 3 \text{ weights} + 4 \text{ biases}) + (2 \times 4 \text{ weights} + 2 \text{ biases}) = 26$$

The four hidden values are **not four extra trainable parameters**. They are results of calculations. Likewise, your input measurements and the two predictions are not parameters. θ is a convenient collective name for the entries of W_1 , b_1 , W_2 , and b_2 .

Connecting this to your two-input transmodel

On your uniform floor, absolute position is unnecessary for predicting displacement. Use velocity and force as inputs, and displacement and final velocity as outputs. A $2 \rightarrow 16 \rightarrow 2$ network contains $(16 \times 2 + 16) + (2 \times 16 + 2) = \mathbf{82 \text{ parameters}}$.

Lesson 6 instead predicts displacement and velocity change, then adds the current velocity. Both conventions work; keep labels and output interpretation consistent. The arithmetic principles in this supplement apply to either.

Prediction and correction are different steps

During a forward pass, the network calculates outputs with fixed current parameters. During training, a surrounding process uses the errors to adjust those parameters.

Step	What happens in your project
1. Supply a batch	Provide input cases with reference motion labels.
2. Predict	Calculate q , h , and the two predicted outputs.
3. Measure error	Calculate a loss comparing predictions with labels, using suitable scales for displacement and velocity errors.
4. Calculate gradients	Backpropagation uses the chain rule to calculate the loss derivatives with respect to the weights and biases.
5. Update	An optimizer changes those parameters using the gradients.

How can an internal neuron receive feedback?

Its output contributes to the final predictions. A change to its weight changes its weighted sum, which can change its activation, which can change the prediction and therefore the loss. The chain rule follows that connected sequence.

hidden weight $\rightarrow q \rightarrow h \rightarrow$ prediction $\rightarrow$ loss

You do not need a “correct h_1 ” label. The final prediction error supplies a gradient through the calculation. Both outputs can contribute feedback to the same hidden neuron.

The division of work: you choose the task, data, architecture, loss, and training setup. Automatic differentiation calculates gradients. The optimizer updates parameters. Your training loop repeats the process.

What training does not promise

Weights normally start from random initialization chosen for the architecture, with biases initialized according to the library or your choice. Training seeks lower loss across examples. It can fail, overfit, or learn simulator mistakes. Good predictions on unseen examples are evidence of usefulness; the existence of many neurons is not.

After training, ordinary prediction keeps the weights fixed. The model does not automatically correct itself without an explicit learning process and a source of feedback.

Lesson 9 develops gradients and optimizer updates. For Lesson 5, your main goal is to explain what is being calculated and which numbers training can change.

Can you explain it without the diagram?

Try these in your notebook before reading the answers. You do not need to write Python to complete them.

1. For the original neuron, use $z = (0, 0, 0.5)$. Calculate q and its ReLU output.
2. Keeping that input, change only its bias from -0.1 to -0.5 . What happens?
3. In the complete network on page 5, increase only the velocity output's coefficient on h_1 from 0.2 to 0.3 . How much does predicted velocity change for that example?
4. Count the parameters in a fully connected $2 \rightarrow 4 \rightarrow 2$ network with biases in both computed layers.
5. Why are hidden values not parameters? Why does a hidden value of zero not mean the cuboid must be at rest?

Answers and reasoning

1. $q = 0.8 \times 0.5 - 0.1 = 0.3$, so $h = 0.3$.
2. $q = 0.4 - 0.5 = -0.1$, so $h = 0$. The bias shifts where this response becomes active.
3. The scaled change is $(0.3 - 0.2) \times 0.7 = 0.07$. With the chosen 1 m/s scale, predicted velocity rises from 0.26 to 0.33 m/s . The position output is unchanged in this forward calculation.
4. $(4 \times 2 + 4) + (2 \times 4 + 2) = 22$ parameters.
5. Hidden values are recalculated from inputs and parameters. A zero value only means that one response is inactive; other neurons and output biases still contribute to predictions. Its meaning is not predefined as a physical state.

Your checkpoint: describe one neuron's calculation, distinguish inputs from parameters and hidden values, follow one full prediction, and explain why the activation matters. If you can do that, Lesson 5's compact equations should now be readable.

Sources and scope

Primary reference: *transmodel_learning_guide.pdf*, Lesson 05, "Inside a small neural network," page 5, prepared 14 September 2026. The original neuron, $3 \rightarrow 4 \rightarrow 2$ structure, 26-parameter count, activation discussion, and checkpoint are expanded here. Lessons 3, 6, 7, and 9 provide related physical, architecture, scaling, and training context. The other toy weights and worked network are new illustrative material; no training results are claimed.

Optional visual companion: [3Blue1Brown: Neural Networks](#). Later implementation reference: [PyTorch: Build the Neural Network](#).